

LAPORAN PRAKTIKUM
SISTEM ADMINISTRASI DAN INFORMASI TERDISTRIBUSI
PERTEMUAN 9
API AUTHENTICATION



Disusun oleh:

Nama : Hanan Fijananto
NIM : 24/538946/SV/24555
Kelas : B2
Dosen Pengampu : Dr.Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT
LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

A Tujuan

1. Mahasiswa dapat melakukan autentikasi pada API
2. Mahasiswa menjalankan sistem register yang dapat membuat token
3. Mahasiswa mampu melakukan login menggunakan token yang ada berdasarkan role yang tersedia

B Dasar Teori

B.1 API Authentication

Autentikasi API adalah proses untuk memvalidasi identitas klien yang mencoba mengakses sumber daya server melalui API. Proses ini memastikan bahwa hanya pengguna atau sistem yang terotorisasi yang dapat berinteraksi dengan API, sehingga melindungi data sensitif dan fungsionalitas dari akses yang tidak sah [1].

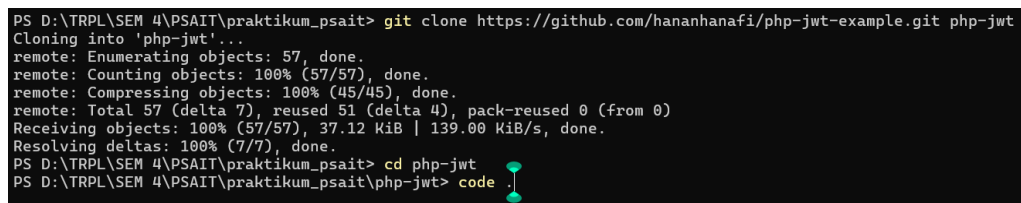
C Hasil dan Pembahasan

Pada praktikum kali ini, dilakukan implementasi autentikasi API menggunakan token untuk mengamankan *endpoint*. Proses ini terdiri dari beberapa tahapan utama, yakni registrasi pengguna baru, proses validasi *login* yang menghasilkan pembuatan token, dan penggunaan token tersebut untuk mengakses bagian sistem yang dilindungi (*protected route*).

C.1 Mempersiapkan source code yang akan digunakan

Buat projek baru dengan nama php-jwt yang diperoleh dari hasil clone repository berikut

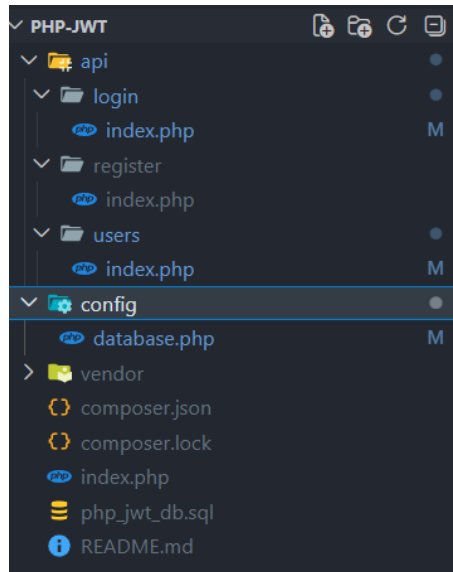
```
1 git clone  
https://github.com/hananhanafi/php-jwt-example.git
```



```
PS D:\TRPL\SEM 4\PSAIT\praktikum_psait> git clone https://github.com/hananhanafi/php-jwt-example.git php-jwt  
Cloning into 'php-jwt'...  
remote: Enumerating objects: 57, done.  
remote: Counting objects: 100% (57/57), done.  
remote: Compressing objects: 100% (45/45), done.  
remote: Total 57 (delta 7), reused 51 (delta 4), pack-reused 0 (from 0)  
Receiving objects: 100% (57/57), 37.12 KiB | 139.00 KiB/s, done.  
Resolving deltas: 100% (7/7), done.  
PS D:\TRPL\SEM 4\PSAIT\praktikum_psait> cd php-jwt  
PS D:\TRPL\SEM 4\PSAIT\praktikum_psait\php-jwt> code .
```

Gambar 1: Clone repository php-jwt-example

Setelah selesai cloning, maka struktur folder dari projek tersebut dapat dilihat pada gambar 2.



Gambar 2: Struktur Direktori Projek

C.2 Konfigurasi Konektivitas Database

Setelah berhasil melakukan cloning, praktikan perlu melakukan konfigurasi database terlebih dahulu. Ubah file database.php sesuai dengan konfigurasi yang digunakan. Sesuaikan host MySQL, nama database yang telah dibuat (jika belum, praktikan dapat membuat database baru dengan nama php_jwt_db terlebih dahulu di phpmyadmin), user database dan passwordnya, dan juga port database secara default port yang digunakan untuk MySQL adalah 3306.

```

class DatabaseService{

    private $db_host = "localhost";
    private $db_name = "php_jwt_db";
    private $db_user = "root";
    private $db_password = "123";
    private $db_port = "3306";
    private $connection;

    public function getConnection(){

        $this->connection = null;
        try{
            $this->connection = new PDO("mysql:host=" . $this->db_host . ";port=" .
$this->db_port . ";dbname=" . $this->db_name, $this->db_user, $this->db_password);
        }catch(PDOException $exception){
            echo "Connection failed: " . $exception->getMessage();
        }
        return $this->connection;
    }
}
?>

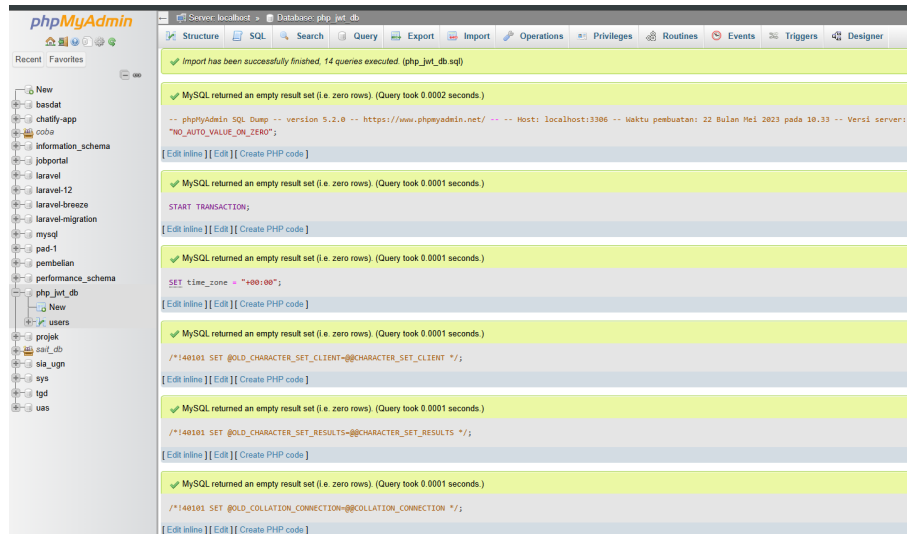
```

Gambar 3: Konfigurasi Konektivitas Database

C.3 Menambahkan tabel ke database

Setelah melakukan konfigurasi database, praktikan perlu menambahkan tabel ke database yang telah dibuat. Tabel yang digunakan pada praktikum ini adalah tabel users. Tabel tersebut dapat dibuat dengan cara mengimport file php_jwt_db.sql yang terdapat pada root projek. Caranya yaitu dengan masuk ke phpmyadmin, pilih database yang telah dibuat, lalu pilih menu import dan pilih file php_jwt_db.sql yang terdapat pada root projek. Setelah itu klik go untuk mengimport tabel users ke database.

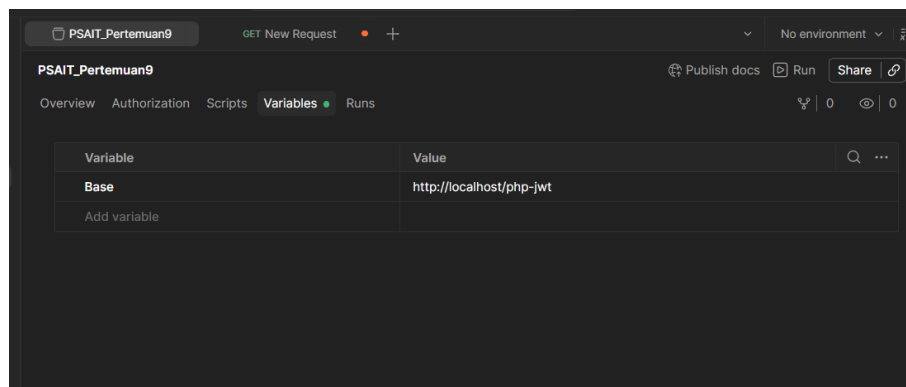
Setelah menekan tombol go, jika berhasil akan terlihat seperti pada gambar 4.



Gambar 4: Hasil import Tabel Users ke Database

C.4 Membuat Konfigurasi API di Postman

Masuk ke Postman, lalu buat variabel baru dengan nama BASE dan isikan dengan `http://localhost/php-jwt`. Variabel ini akan digunakan untuk mempermudah dalam melakukan request ke API yang telah dibuat.

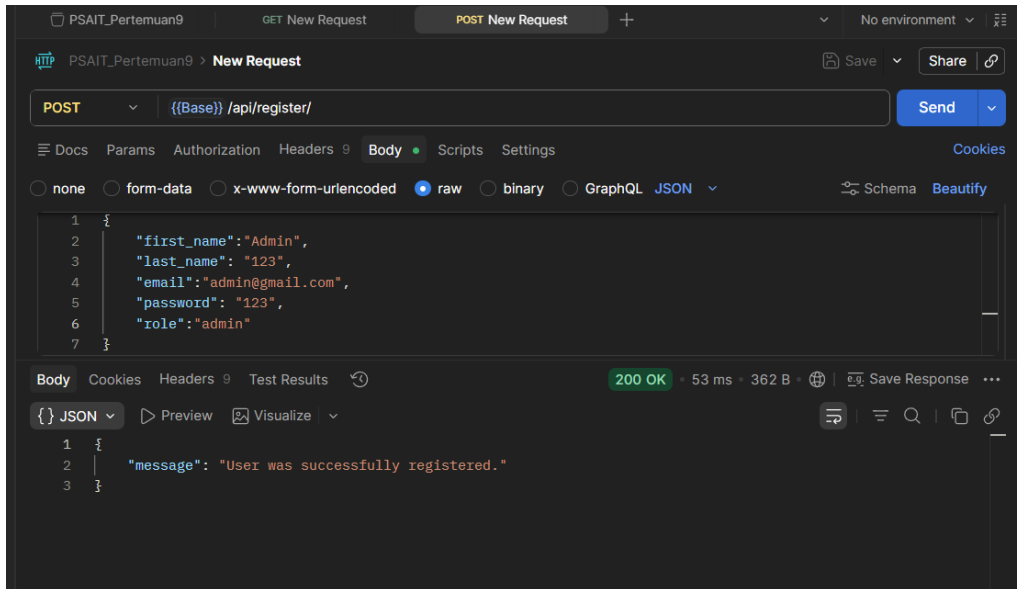


Gambar 5: Membuat Variabel BASE di Postman

C.5 Uji register pada Postman

Tahap pertama adalah melakukan register untuk membuat akun baru. Praktikan dapat melakukan register dengan cara melakukan request POST ke endpoint

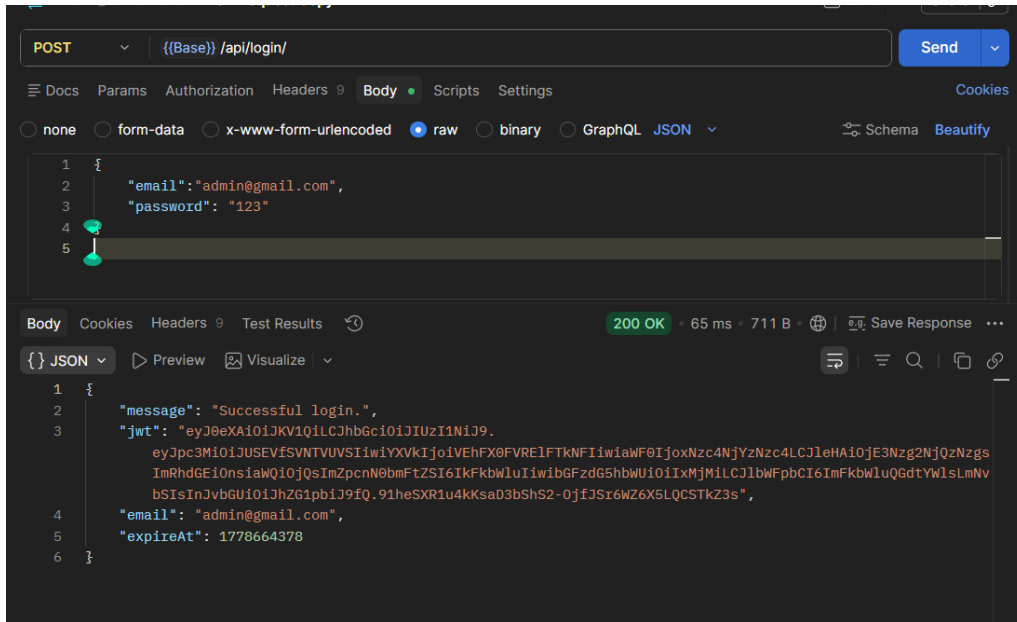
api/register dengan body berupa JSON yang berisi nama, email, password, dan role. Setelah itu klik send untuk melakukan request ke API. Jika berhasil, maka akan muncul response dengan status success dan data user yang telah dibuat seperti pada gambar 6.



Gambar 6: Uji Register pada Postman

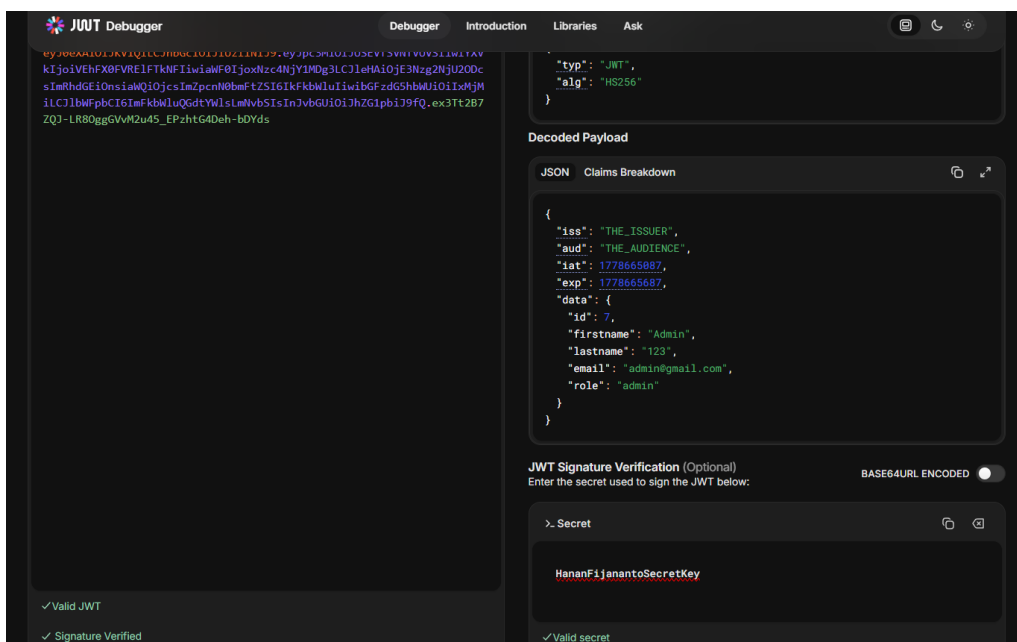
C.6 Uji login pada Postman

Tahap kedua adalah melakukan login untuk mendapatkan token. Praktikan dapat melakukan login dengan cara melakukan request POST ke endpoint api/login dengan body berupa JSON yang berisi email dan password. Setelah itu klik send untuk melakukan request ke API. Jika berhasil, maka akan muncul response dengan status success dan data user beserta token yang telah dibuat seperti pada gambar 7.



Gambar 7: Uji Login pada Postman

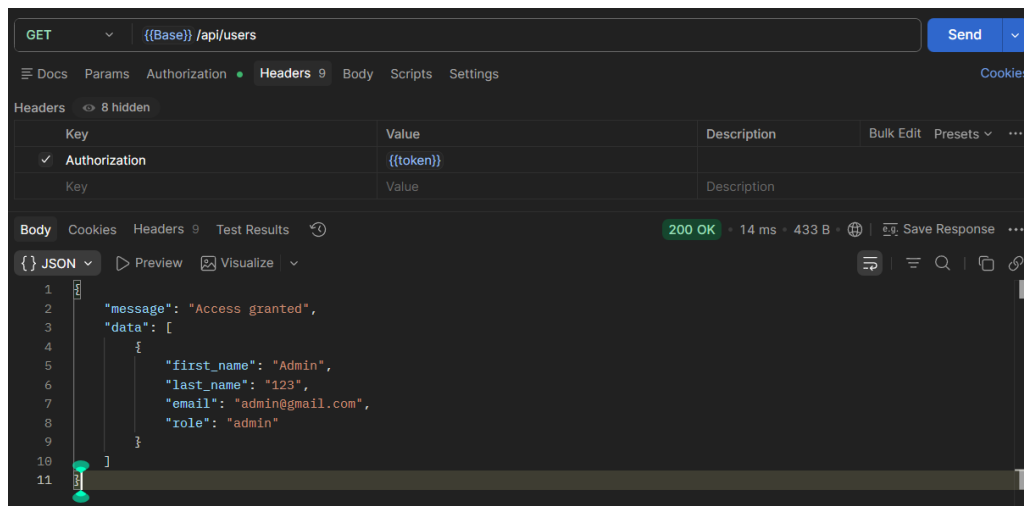
Setelah mendapat token jwt, mahasiswa dapat melakukan debugging isi dari token tersebut dengan menggunakan website jwt.io. Pada website tersebut, mahasiswa dapat memasukkan token yang telah didapatkan pada bagian encoded, lalu akan muncul isi dari token tersebut pada bagian decoded seperti pada gambar 8.



Gambar 8: Debugging Token JWT

C.7 Uji mengambil data dengan role admin

Setelah berhasil melakukan login, praktikan dapat menggunakan token yang telah didapatkan untuk mengakses endpoint yang dilindungi. Pada praktikum ini, terdapat endpoint `api/users` yang hanya dapat diakses oleh user dengan role admin. Praktikan dapat melakukan request GET ke endpoint `api/users` dengan menyertakan token pada header `Authorization`. Jika berhasil, maka akan muncul response dengan status success dan data yang hanya dapat diakses oleh user dengan role admin seperti pada gambar 9.



Gambar 9: Uji Mengambil Data dengan Role Admin

D Tugas dan Analisis

Mahasiswa diberi 5 soal terkait dengan implementasi autentikasi API yang telah dilakukan. Berikut adalah soal-soal beserta hasil analisisnya:

1. Apa yang terjadi ketika melakukan request akses data user menggunakan akun dengan role admin dan jwt token yang valid?

Jawab:

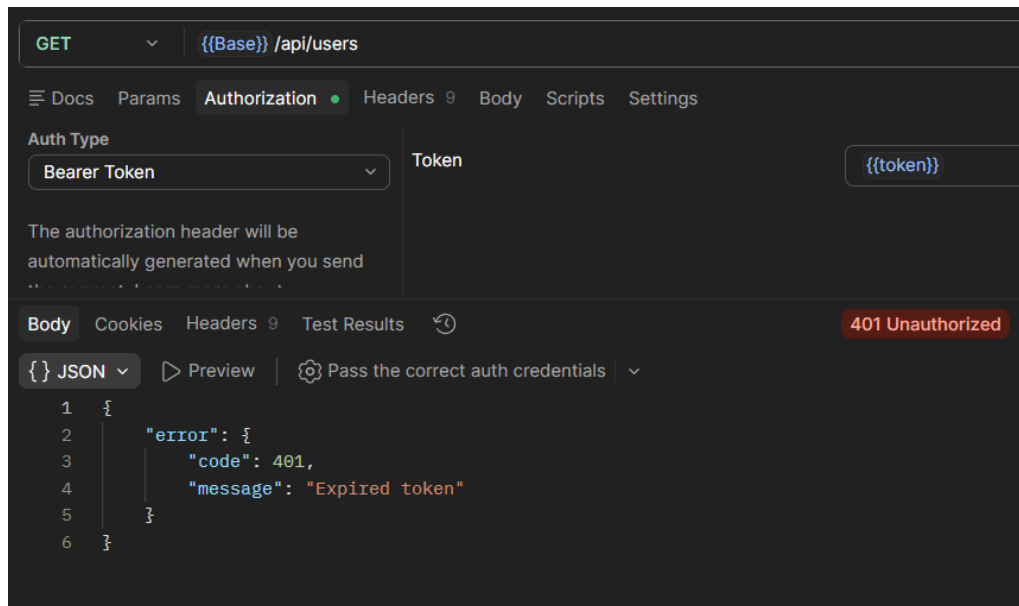
Ketika melakukan request akses data user menggunakan akun dengan role admin dan jwt token yang valid, maka request tersebut akan berhasil dan

server akan mengembalikan data user yang diminta. Hal ini terjadi karena token yang valid menunjukkan bahwa pengguna telah terautentikasi dengan benar dan memiliki hak akses yang sesuai untuk mengakses data tersebut. hal tersebut dapat dilihat pada gambar 9 yang menunjukkan bahwa request berhasil dan data user berhasil diakses menggunakan token yang valid.

2. Apa yang terjadi ketika melakukan request akses data user menggunakan akun dengan role admin dan jwt token yang tidak valid?

Jawab:

Ketika melakukan request akses data user menggunakan akun dengan role admin dan jwt token yang tidak valid, maka request tersebut akan gagal dan server akan mengembalikan response dengan status error. Hal ini terjadi karena token yang tidak valid menunjukkan bahwa pengguna tidak terautentikasi dengan benar atau token tersebut telah kedaluwarsa, sehingga server menolak akses ke data user. Response error yang diterima biasanya berupa pesan yang menyatakan bahwa token tidak valid atau akses ditolak, seperti pada gambar 10 yang menunjukkan bahwa request gagal karena token tidak valid.

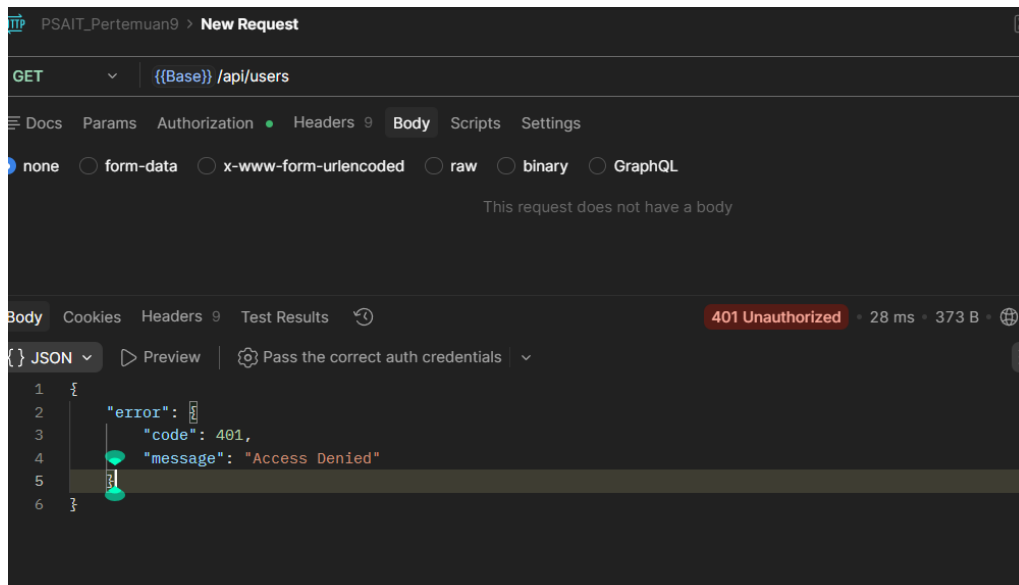


Gambar 10: Uji Mengakses Data dengan Token Tidak Valid

3. Apa yang terjadi jika akun dengan role user biasa (bukan admin) mengakses data user?

Jawab:

Ketika akun dengan role user biasa (bukan admin) mencoba mengakses data user, maka request tersebut akan gagal dan server akan mengembalikan response dengan status error. Hal ini terjadi karena role user biasa tidak memiliki hak akses yang cukup untuk mengakses data user, sehingga server menolak akses tersebut. Response error yang diterima dapat dilihat pada gambar 11.

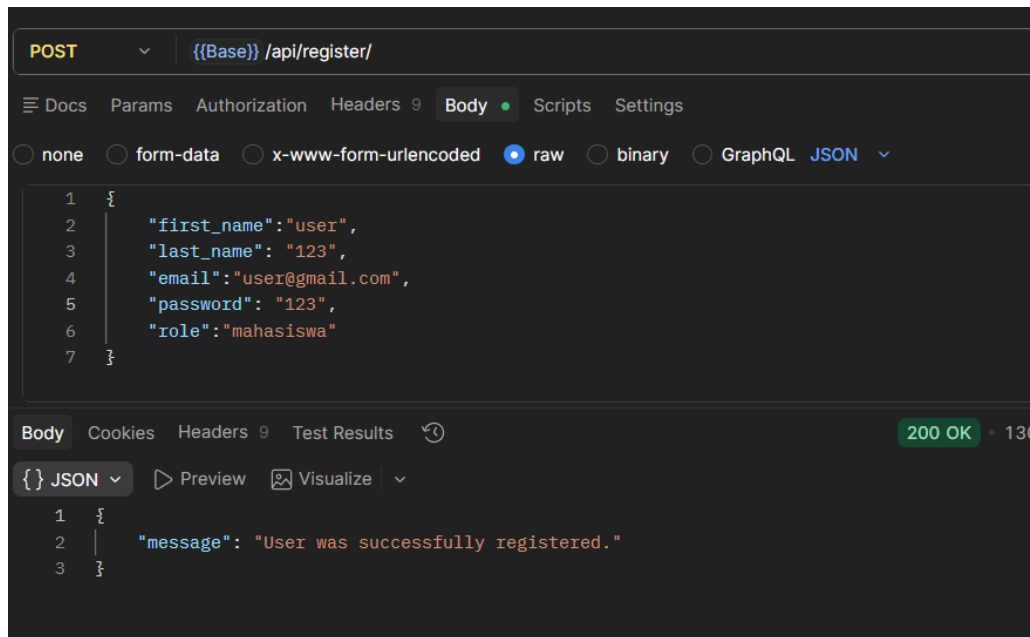


Gambar 11: Uji Mengakses Data dengan Role User Biasa

4. Apa yang terjadi jika melakukan register dengan menginput role selain admin dan user biasa?

Jawab:

Jika melakukan register dengan menginput role selain admin dan user, maka proses registrasi akan tetap berhasil karena di dalam file `register/index.php` tidak terdapat validasi untuk membatasi jenis role yang diinputkan. Dapat dilihat pada gambar 12, bahwa role mahasiswa berhasil dibuat meskipun tidak sesuai dengan role yang telah ditentukan (admin dan user biasa).



Gambar 12: Uji Register dengan Role Mahasiswa

5. Apa yang terjadi jika akun dengan role selain admin dan user biasa mengakses data user?

Jawab:

Jika akun dengan role selain admin dan user biasa mencoba mengakses data user, maka sistem yang dibuat tidak memiliki validasi untuk membatasi jenis role yang dapat mengakses data user. Oleh karena itu, sistem tersebut memiliki mekanisme authorization yang kurang baik.

E Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi autentikasi API menggunakan token JWT dapat memberikan keamanan yang lebih baik dalam mengakses sumber daya server. Dengan menggunakan token, hanya pengguna yang terautentikasi dengan benar dan memiliki hak akses yang sesuai yang dapat mengakses data atau fungsionalitas tertentu. Selain itu, penggunaan token juga memungkinkan untuk mengelola sesi pengguna dengan lebih efisien, karena token dapat memiliki masa berlaku tertentu dan dapat diperbarui sesuai kebutuhan. Namun, penting untuk memastikan bahwa implementasi autentikasi API dilakukan dengan benar dan aman, termasuk penggunaan algoritma enkripsi yang kuat untuk token dan validasi yang tepat untuk mencegah akses yang tidak sah.

Dengan adanya validasi role, sistem dapat membatasi akses ke sumber daya tertentu hanya untuk pengguna dengan hak akses yang sesuai, sehingga meningkatkan keamanan dan kontrol terhadap data yang sensitif. Namun, perlu diingat bahwa implementasi autentikasi API harus dilakukan dengan hati-hati dan mengikuti praktik terbaik untuk memastikan keamanan yang optimal.

F Daftar Pustaka

- [1] A. Prasetyo, “Apa Itu Autentikasi API dan Mengapa Penting? — echoapi.id,” <https://www.echoapi.id/blog/apa-itu-autentikasi-api-dan-mengapa-penting/>, 2025, [Accessed 19-05-2026].